

COMP3161/COMP9164: Preliminaries

Gabriele Keller

Johannes Åman Pohjola*

September 20, 2022

1 Introduction

As we are going to discuss and reason about properties of various programming languages and language features, we need a formal meta-language which allows us to make statements about these properties. We need to specify the grammar of a language, the static semantics (often in form of typing and scoping rules) and dynamic semantics. Fortunately, it turns out that a single formalism, inductive definitions built on inference rules, is sufficient.

2 Judgements and Inference Rules

A **judgement** is simply a statement that a certain property holds for a specific object,¹ for example:

- $3 + 4 * 5$ is a valid arithmetic expression
- the string "aba" is a palindrome
- 0.43423 is a floating point value

Judgements are not unlike predicates you might know from Predicate Logic. We write

$$a \ S$$

for a judgement of the form *The property S holds for object a* . In predicate logic, this is usually written differently, as $S(a)$. However, we will see later that for our purposes, it is much more convenient to write it in the above given post-fix notation. Alternatively, we can interpret S as a set of objects with a certain property, and read the judgement $a \ S$ as: a is an element of the set S . Some examples of judgements and how to read them are:

- $5 \ even$
 - 5 is even, or
 - 5 is an element of the set of even numbers
- $3 + 4 * 5 \ expr$
 - $3 + 4 * 5$ is a syntactically correct expression, or
 - $3 + 4 * 5$ an element of the set containing all syntactically correct expressions
- $0.43423 \ float$

*Minor edits and clarifications. Gabriele is the main author.

¹More generally, a relationship between a number of objects holds. For now, we just look at statements about a single object.

- 0.43423 is a floating point value
- 0.43423 is an element of the set of floating points values

Judgements by themselves would be boring and fairly useless. Most interesting sets have an infinite number of elements, and to define such a set it would obviously be impossible to explicitly list them all using simple judgements. Luckily, the sets we are interested in are not random collections of objects: they can be systematically defined by so-called **inference rules**.

Inference rules allow us to combine judgements to obtain new judgements. They have the following general form:

If judgements J_1 , and J_2 , and $\dots$ and J_n are **inferable**, then the judgement J is **inferable**

and are usually given in the following standard form:

$$\frac{J_1 \ J_2 \ \dots \ J_n}{J}$$

where the judgements J_1 to J_n are called **premises**, and J is called a **conclusion**. An inference rule does not have to have premises, it can consist of a single conclusion. Such inference rules are called **axioms**. But let us have a look at a concrete example now.

We start by defining some simple properties over the set of natural numbers (Nat). For simplicity reasons, we represent them as 0, ($s\ 0$), ($s\ (s\ 0)$), ($s\ (s\ (s\ 0))$) for 0, 1, 2, 3, and so on (s here stands for *successor*). So, first of all, how can we define Nat itself using inference rules? Listing all the element of Nat would be equivalent to including an axiom for each number:

$$\begin{array}{c} \overline{0\ Nat} \\ \overline{(s\ 0)\ Nat} \\ \overline{(s\ (s\ 0))\ Nat} \\ \vdots \end{array}$$

Apart from the first rule, all the rules have the form

$$\overline{(s\ x)\ Nat}$$

where $x\ Nat$ has been established by the previously listed axiom. In words, we have

1. 0 is in Nat , and
2. for all x , if x is in Nat , then $(s\ x)$ in Nat

which can be translated directly into the following two inference rules:

$$\overline{0\ Nat} \qquad \frac{x\ Nat}{\overline{(s\ x)\ Nat}}$$

where x is a variable that can be instantiated to any term. In the same way, we can define the sets *Even* and *Odd*:

$$\overline{0\ Even} \qquad \frac{x\ Even}{\overline{(s\ (s\ x))\ Even}} \qquad \overline{(s\ 0)\ Odd} \qquad \frac{x\ Odd}{\overline{(s\ (s\ x))\ Odd}}$$

Rules do work in two ways: we can use them to define a property, but we can also use them to show that a judgement is valid. How does it work with inference rules? Assume we want to show that some judgement J is valid. We have to look for a rule which has J as a conclusion. If this rule is an axiom, we are already done. If not, we have to show that all of its premises are

valid by recursively applying the same strategy to all of them. For example, we can show that $(s(s\ 0))\text{Even}$, since

$$\frac{0\ \text{Even}}{(s(s\ 0))\ \text{Even}}$$

and

$$\overline{0\ \text{Even}}$$

As the last rule is an axiom, there are no premises left to prove, and we are done.

Similarly, we can show that $0 + 1 + 1 + 1 + 1\ \text{Even}$. An alternative and often quite convenient way to write inference proofs is to stack the rules we apply together and draw a “proof tree” — in our example, more a proof stack, since each rule has just a single premise.

$$\frac{\frac{\overline{(s\ 0)\ \text{Odd}}}{(s(s(s\ 0)))\ \text{Odd}}}{(s(s(s(s(s\ 0)))))\ \text{Odd}}$$

Note that inference works on a purely syntactic basis. Given the rules above, we are not able to prove $2\ \text{Even}$, even though we can show that $s(s(0))\ \text{Even}$, and we know that $s(s(0))$ is equal to 2. We cannot apply that knowledge, since we have no rule which tells us it is ok to do so. We just mechanically manipulate terms according to the given rules.

Let us look at a slightly more interesting example: we want to define the language M which contains all expressions of properly matched parentheses (and no other characters):²

$$M = \{\epsilon, (), ()(), ()()(), \dots, (()), ((())), \dots, ()(), ()()(), \dots\}$$

Again, let us start by describing the rules in (semi-)natural language. There are basically two ways to “legally” combine parentheses: we can either nest them, or concatenate them:

1. The empty string (denoted by ϵ) is in M
2. If s_1 and s_2 are in M , then $s_1 s_2$ is in M
3. If s is in M , then (s) is in M

Again, these rules can be directly translated into inference rules:

$$\begin{aligned} (1) \quad & \frac{}{\epsilon\ M} \\ (2) \quad & \frac{s_1\ M \quad s_2\ M}{s_1 s_2\ M} \\ (3) \quad & \frac{s\ M}{(s)\ M} \end{aligned}$$

How can we show that $()(())$ is in M ? As we did previously, we check if there is a rule whose conclusion matches the judgement we want to infer. If we apply Rule (2), we have to show that both $()$ and $(())$ are in M . Since $() = (\epsilon)$, we can apply Rule (3), and only have to show that ϵ is in M (Rule (1)). By applying Rule (3) in the same way, we can show that $(())$ is in M as well, and we are done. This argument can be captured in the following proof tree:

$$\frac{\frac{\overline{\epsilon\ M}^{(1)}}{() \ M}^{(3)} \quad \frac{\frac{\overline{\epsilon\ M}^{(1)}}{() \ M}^{(3)}}{()() \ M}^{(2)}}{() (()) \ M}^{(2)}$$

² ϵ represents the empty string

Finding a proof tree is not always as straightforward as it seems. Sometimes, multiple rules may be applicable to a judgement, or a single rule may be applicable in more than one way. Some of these may be dead ends.

For example, instead of applying Rule (2) above, we could also have tried Rule (3). The latter rule has only a single premise, so we only have to prove that $)()$ is in M . The only rule that's applicable now is Rule (2), and we could apply it in several different ways resulting in different premises:

$$\begin{array}{l} \text{or} \quad \frac{) M \quad () M}{) () M} \\ \text{or} \quad \frac{) (M \quad () M}{) () M} \\ \text{or} \quad \frac{) ((M \quad) M)}{) () M} \end{array}$$

In the first application, we end up with the premise: $)$ is in M . But there is no rule which we can apply to get rid of it. This is not that surprising, since M should only contain expressions of properly matched parentheses, and neither $)$ nor $)()$ are such. So, by choosing the wrong rule, or applying the right rule in the wrong way—for example, splitting $()()$ up into $()()$ and $)$ —we can easily end up with premises that are not actually valid, reaching a dead end. In our example, this was not hard to see. It can be extremely difficult to decide which rule to apply and how, without some background knowledge about the objects and properties, as there might be an infinite number of possibilities. This is one reason why it is not possible to write a program which automatically derives judgements, that is guaranteed to find such a derivation if it exists.³ It is, however, possible to write semi-automatic theorem provers, which come up with proofs in cases where it is fairly standard, and rely on user input otherwise.

2.1 Derivable and Admissible Rules

Consider what would happen if we added the following rule to the language M :

$$(4) \quad \frac{s M}{((s)) M}$$

Does this change the set M in any way? That is, is there a string s for which we can infer $s M$ if we use (4), but not otherwise? This doesn't seem very likely: the rule just says that, if a string s is in M it is ok to add two pairs of matching parentheses. Since we already had a rule which allows us to add one pair of parentheses, we can just apply this rule twice and achieve the same effect:

$$\frac{\frac{s M}{(s) M}}{((s)) M}$$

This means that Rule (4) is **derivable** from the existing rules.

In all the previous rules, the strings in the premises were simpler than the string in the conclusion. The following rule is different in this respect:

$$(5) \quad \frac{() s M}{s M}$$

Interestingly, although Rule (5) does not introduce any new strings to M , the rule is also not derivable from any of the existing rules. Such a rule is called **admissible**.

³If it was possible, mathematicians and theoretical computer scientists would be out of a job.

2.2 Inductive Definitions

A set of inference rules R defining a set A is called an **inductive definition** of A , if the set of valid judgements $s A$ are precisely those judgements that can be derived using R . All the examples we have discussed are inductive definitions.

Not all sets can be defined using inductive definitions. For example, while natural numbers are one of the standard examples of such sets, real numbers cannot be defined in such a way.

2.3 Judgements and Relations

So far, we defined a judgement to be a statement about a property of an object. We can generalise this definition to relation between a number of objects. Consider the following inductive definition of the relation “ $a < b$ ” on natural numbers. For convenience reasons, we choose an infix notation here:

$$\frac{n \text{ Nat}}{0 < (s n)}$$

$$\frac{n < m}{(s n) < (s m)}$$

As before, we can also view this as an inductive definition of a set. In this case, a set of pairs, where $(a, b) \in (<)$ if and only if a is less than b .

3 Rule Induction

Natural deduction—the process of deriving judgements from inference rules outlined above—is sometimes not powerful enough by itself. For example, although we can see that the Rule (5) in Section 2.1 is valid for every string s in M , we cannot show this by simply combining the existing rules. We will therefore introduce another proof technique here, called **induction**. You will probably know induction over natural numbers and structural induction from mathematics and previous courses. Both are special cases of a more general induction principle called **rule induction**.

Let us go back to our previous example set M of properly matched parentheses. The rules 1 to 3 provide an inductive definition of M :

$$(1) \quad \frac{}{\epsilon M}$$

$$(2) \quad \frac{s_1 M \quad s_2 M}{s_1 s_2 M}$$

$$(3) \quad \frac{s M}{(s) M}$$

Now, let us assume we want to prove some property P of the strings in M , that is: show that if $s M$ then $s P$. Since we know that there is a derivation for each $s M$, we only need to show that:

- ϵP
- if $s_1 P$ and $s_2 P$, then $s_1 s_2 P$
- if $s P$, then $(s) P$

Which in essence corresponds to the original rules only with M replaced by P . For example, if we want to show that all s in M have the same number of opening and closing brackets, we need to prove the following statements (where *open* and *close* denotes the number of opening and closing brackets, respectively):

1. $open(\epsilon) = close(\epsilon)$

Proof: $open(\epsilon) = 0 = close(\epsilon)$

2. if $open(s_1) = close(s_1)$ and $open(s_2) = close(s_2)$ then $open(s_1 s_2) = close(s_1 s_2)$

For the proof, we assume that the statements corresponding to the judgements in the premises of the rules hold. These assumptions are called the **induction hypothesis**.

- Induction Hypothesis 1: $open(s_1) = close(s_1)$
- Induction Hypothesis 2: $open(s_2) = close(s_2)$

Proof: $open(s_1 s_2) = open(s_1) + open(s_2) = close(s_1) + close(s_2) = close(s_1 + s_2)$

3. if $open(s) = close(s)$ then $open((s))$

- Induction Hypothesis 1: $open(s) = close(s)$

Proof:

$$\begin{aligned}
& open((s)) \\
&= \{\text{property of } open\} \\
& \quad open() + open(s) + open() \\
&= \{\text{property of } open\} \\
& \quad 1 + open(s) + 0 \\
&= \{\text{Induction Hypothesis, Arithmetic}\} \\
& \quad 1 + close(s) + 0 \\
&= \{\text{property of } close\} \\
& \quad close() + close(s) + close() \\
&= \{\text{property of } close\} \\
& \quad close((s))
\end{aligned}$$

In the proof, we used the rules of arithmetic, and that properties of $close$ and $open$, such as $open() = close() = 1$, and $open(s) = close(s) = 0$. In a fully formal proof, we would also need a formal definition of these two functions.

3.1 Ambiguity

The definition of M , although correct, has an undesirable property: for any string in M , we do not have just one derivation, but an infinite number of possible derivations, since any string s can be split into ϵ and s by applying Rule (2), and then Rule (1) to get rid of ϵ . However, this derivation step is completely unnecessary.

Fortunately, we can come up with an alternative set of rules, where we have exactly one derivation for each string in the set. We can interpret each string as a possibly empty list (L) of non-empty parenthesis expressions (N) according to the following inference rules:

$$\begin{aligned}
(1) \quad & \frac{}{\epsilon L} \\
(2) \quad & \frac{s_1 N \quad s_2 L}{s_1 s_2 L} \\
(3) \quad & \frac{s L}{(s) N}
\end{aligned}$$

The interesting point here is that L and N are defined in terms of each other: we have a mutually recursive definition.

Let us look at one more example of an ambiguous definition: simple arithmetic expressions, given here both in EBNF form and as inductive definition using inference rules:

The EBNF

$$Expr \rightarrow int \mid (Expr) \mid Expr + Expr \mid Expr * Expr$$

describes the same language as the following set of inference rules. We assume that *int* represents an integer constant, and that integer constant expressions are already defined in some suitable way:

$$\frac{i \text{ int}}{i \text{ Expr}} \quad \frac{e \text{ Expr}}{(e) \text{ Expr}} \quad \frac{e_1 \text{ Expr} \quad e_2 \text{ Expr}}{e_1 + e_2 \text{ Expr}} \quad \frac{e_1 \text{ Expr} \quad e_2 \text{ Expr}}{e_1 * e_2 \text{ Expr}}$$

Although in this case, there is not an infinite number of possible derivations for each expression, every expression which contains more than a single arithmetic operation can still be derived in more than one way:

$$\frac{\frac{1 \text{ int}}{1 \text{ Expr}} \quad \frac{2 \text{ int}}{2 \text{ Expr}} \quad \frac{3 \text{ int}}{3 \text{ Expr}}}{\frac{1 + 2 \text{ Expr} \quad 3 \text{ Expr}}{1 + 2 * 3 \text{ Expr}}}$$

$$\frac{\frac{1 \text{ int}}{1 \text{ Expr}} \quad \frac{2 \text{ int}}{2 \text{ Expr}} \quad \frac{3 \text{ int}}{3 \text{ Expr}}}{\frac{1 \text{ Expr} \quad 2 * 3 \text{ Expr}}{1 + 2 * 3 \text{ Expr}}}$$

Although both derivations are correct with respect to the rules given, the second derivation is more appropriate for an arithmetic expression, as it decomposes the expression first into two summands. We give an alternative definition, which takes precedence and associativity of the operators into account:

$$\frac{e_1 \text{ SExpr} \quad e_2 \text{ PExpr}}{e_1 + e_2 \text{ SExpr}} \quad \frac{e \text{ PExpr}}{e \text{ SExpr}}$$

$$\frac{e_1 \text{ PExpr} \quad e_2 \text{ FExpr}}{e_1 * e_2 \text{ PExpr}} \quad \frac{e \text{ FExpr}}{e \text{ PExpr}}$$

$$\frac{e \text{ SExpr}}{(e) \text{ FExpr}} \quad \frac{i \text{ int}}{i \text{ FExpr}}$$

The unambiguous grammar is, again, much more complicated than the original grammar, even for such a simple language. This is not surprising, as it contains additional structural information. For programming languages, ambiguous grammars are problematic, as they may allow different interpretations of programs, and are therefore usually avoided.

3.2 Simultaneous Induction

How can we apply the principle of rule induction to mutually recursive definitions like those of *L* and *SExpr*? In most cases, we have to generalise the proof goal. For example, if we want to show the following property for all *e* in *SExpr*: that *e SExpr* implies *e PExpr*. By the principle of rule induction we have to show:

- Assuming

- $e = e_1 + e_2$
- $e_1 \text{ SExpr}$
- $e_2 \text{ PExpr}$

and the Induction Hypothesis

- $e_1 P$

show that $e_1 + e_2 P$

- under the assumption that

- $e \text{ PExpr}$

show that $e P$

The problem is that, since we are only trying to show something about SExpr , the induction hypothesis does not say anything about e_2 (we only know that $e_2 \text{ PExpr}$). The case for the second rule will have a similar problem for e . In most cases, this is not enough to prove anything. The solution is often to try and prove a more general statement instead, which leads to a stronger induction hypothesis. If we try to show, for instance, that $e \text{ SExpr}$ or $e \text{ PExpr}$ or $e \text{ FExpr}$ implies $e P$, we have more cases to cover on one hand (one for each inference rule which has PExpr , SExpr or FExpr in the conclusion), on the other hand the induction hypothesis now covers all the premises in the rules:

- Assuming

- $e = e_1 + e_2$
- $e_1 \text{ SExpr}$
- $e_2 \text{ PExpr}$

and the Induction Hypothesis

- $e_1 P$
- $e_2 P$

show that $e_1 + e_2 P$

- under the assumption that

- $e \text{ PExpr}$

and the Induction Hypothesis

- $e P$

show that $e P$ (trivial)

- under the assumption that

- $e = e_1 * e_2$
- $e_1 \text{ SExpr}$
- $e_2 \text{ PExpr}$

and the Induction Hypothesis

- $e_1 P$
- $e_2 P$

show that $e_1 * e_2 P$

- ...

4 Examples

4.1 Boolean Expressions

As another example, consider boolean expressions. For simplicity, we only include three operators for now: $\wedge$, $\vee$, and $\neg$, the constants *True* and *False*, and parentheses. Our first attempt at defining a set of inference rules to characterise boolean expressions might look as follows:

$$\begin{array}{c} \frac{}{\text{True } BExpr} \quad \frac{}{\text{False } BExpr} \quad \frac{e \ BExpr}{\neg e \ BExpr} \\[10pt] \frac{e \ BExpr}{(e) \ BExpr} \quad \frac{e_1 \ BExpr \quad e_2 \ BExpr}{e_1 \wedge e_2 \ BExpr} \quad \frac{e_1 \ BExpr \quad e_2 \ BExpr}{e_1 \vee e_2 \ BExpr} \end{array}$$

Unfortunately, with this set of rules, we have the same problem we had with our rules for arithmetic expressions. Even though they inductively define the set of boolean expressions, they are ambiguous and do not reflect associativity and precedence of the operators. So, we need to come up with an alternative definition. The operator $\neg$ has the highest precedence, $\vee$ the lowest, and both $\wedge$ and $\vee$ are right associative. The solution is also similar to the solution for arithmetic expressions. First, we need rules to define the subset of boolean expressions which can be arguments of the operator with the highest precedence, negation. These can only be atomic expressions (constants), any expression in parentheses, or such an expression preceded by negation. Let's call this subset *NBExpr*. We call the boolean expressions we generate with the new rules *BExpr*.

$$\frac{}{\text{True } NBExpr} \quad \frac{}{\text{False } NBExpr} \quad \frac{e \ NBExpr}{\neg e \ NBExpr} \quad \frac{e \ BExpr}{(e) \ NBExpr}$$

The rules for the operators $\wedge$ and $\vee$ correspond to those for addition and multiplication. Since the operators are right associative, the expression on the left side can only be an expression with stronger cohesion than the one on the right hand side. For the $\wedge$ operator, it has to be *NBExpr*.

$$\frac{e_1 \ NBExpr \quad e_2 \ ABExpr}{e_1 \wedge e_2 \ ABExpr} \quad \frac{e_1 \ ABExpr \quad e_2 \ BExpr}{e_1 \vee e_2 \ BExpr}$$

And finally we need rules to express the fact that $NBExpr \subseteq ABExpr \subseteq BExpr$

$$\frac{e \ NBExpr}{e \ ABExpr} \quad \frac{e \ ABExpr}{e \ BExpr}$$

4.2 More Arithmetic

We can define addition on natural numbers inductively as a relation between three numbers n, m , and k , where $n + m = k$.

$$\begin{array}{ll} (Add1) & \frac{n \ Nat}{0 + n = n} \\[10pt] (Add2) & \frac{n + m = k}{(s \ m) + n = (s \ k)} \end{array}$$

The rule

$$\frac{n \text{ Nat}}{n + 0 = n}$$

is not derivable from *Add1* and *Add2*, but it is admissible. You can show that this is the case using induction over natural numbers. Similarly, for the following rule:

$$\frac{n + m = k}{m + n = k}$$

Acknowledgements These lectures draw on material from the draft book on *Programming Languages: Theory and Practice*, Robert Harper, and contain examples taken from Frank Pfenning's lecture notes for the CMU course *Foundations of Programming Languages*.